



Today's agenda

↳ Intro

↳ Types of graph

↳ Storage

↳ BFS (level order) +1



AlgoPrep

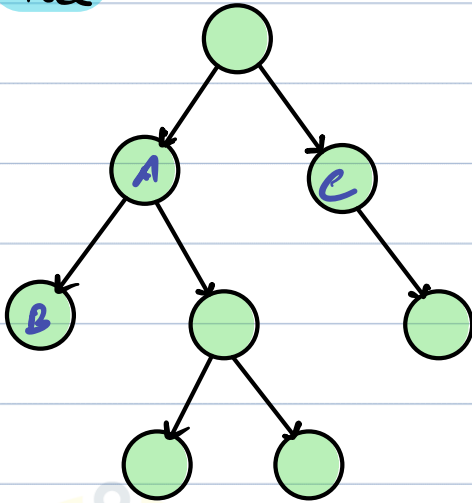


Intro

↳ Graph: Connection of nodes and edges.

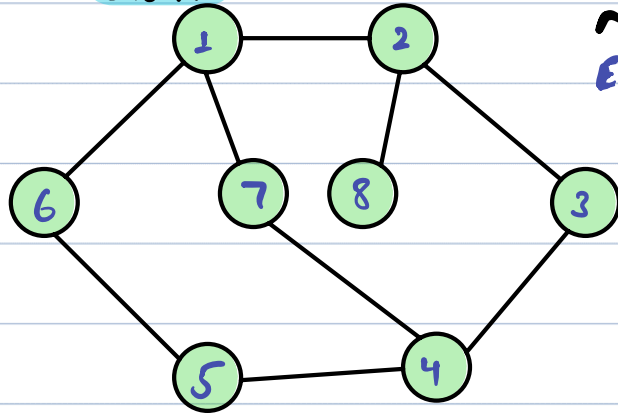
1. Tree

$N=8$
 $E=7$



Graph

$N=8$
 $E=9$



→ diffⁿ betⁿ graph and tree

1) Any number of connections are allowed in graph, so there is no hierarchy.

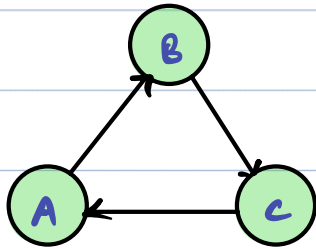
↓
graph can have cycles

2) Any direction movement is allowed in graph.

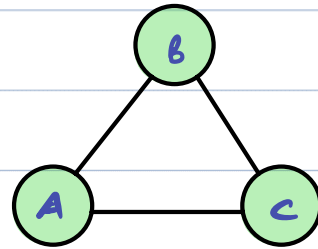


* Classification of graphs

Case I: Based on types of edges.

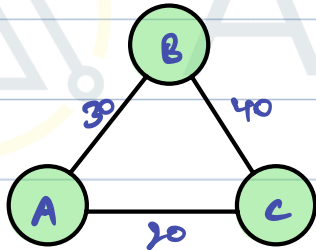


↳ directed graph
(insta followers)

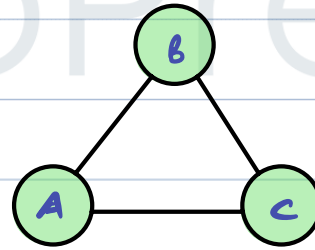


↳ undirected graph
(linkedin connect)

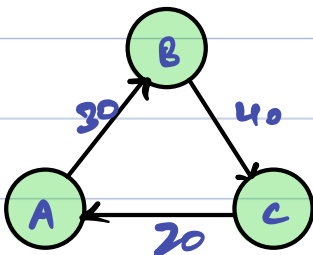
Case II: Based on weight of edge



↳ weighted graph
(maps)

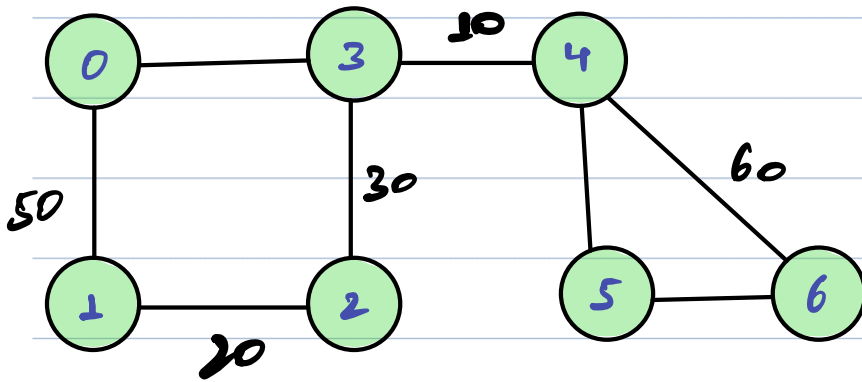


↳ unweighted graph
(^^)



→ weighted directed graph

Storing a graph



undirected graph
input
nodes
 $N=7$
 $m=8$ edges

0	3	4	10
1	1	2	20
2	2	3	30
3	4	6	60
4	0	1	50
5	4	5	
6	5	6	
7	0	3	

① Adjacency matrix representation

Graph

Nodes

	0	1	2	3	4	5	6
0	0	1	0	1	0	0	0
1	1	0	1	0	0	0	0
2	0	1	0	1	0	0	0
3	1	0	1	0	1	0	0
4	0	0	0	1	0	1	1
5	0	0	0	0	1	0	1
6	0	0	0	0	1	1	0

Nodes

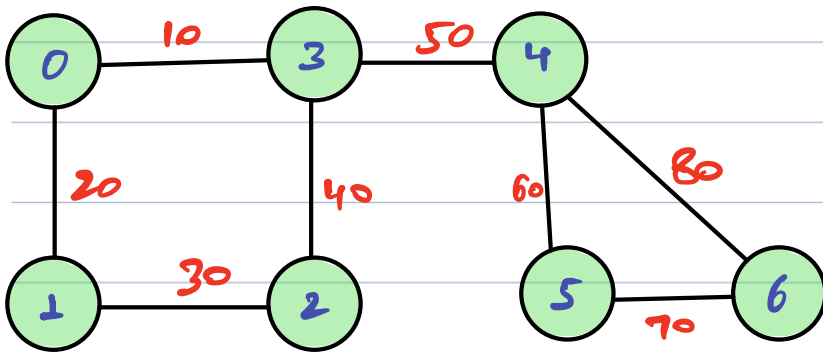
7x7

0 → disconnected
1 → Connected

↳ a lot of space wastage.



② adjacency list representation



edges $N=7$ $M=8$ $N=V$ $M=E$

undirected graph

$i \rightarrow$	u	v	wt
0	3	4	10
1	1	2	20
2	2	3	30
3	4	6	60
4	0	1	50
5	4	5	
6	5	6	
7	0	3	80

Pair

```

List<List<Integer>> graph = new ArrayList<>();
for(int i=0; i<N; i++) {
    graph.add(new ArrayList<>());
}
    
```

$u=3$ $v=4$

graph

0	
1	
2	
3	
4	
5	
6	

`graph.get(3).add(4)`

for wt:

class Pair {

int v;

int wt;

}



//PSuedo Code

List<List<Integer>> Construction (int ^{node}N, int ^{edges}M, int edges[M][2]) {

List<List<Integer>> graph = new ArrayList<>();

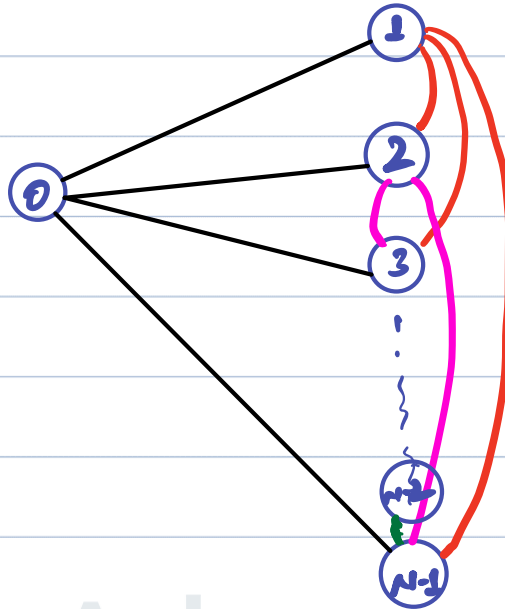
for (int i=0; i<N; i++) {
graph.add(new ArrayList<>());
}

for (int i=0; i<M; i++) {
int u = edges[i][0];
int v = edges[i][1];
graph.get(u).add(v);
graph.get(v).add(u);
}

return graph;

}

Q) Find max no. of edges possible if N nodes are present in Graph.



$$(N-1) + (N-2) + (N-3) + \dots + 1$$

↓

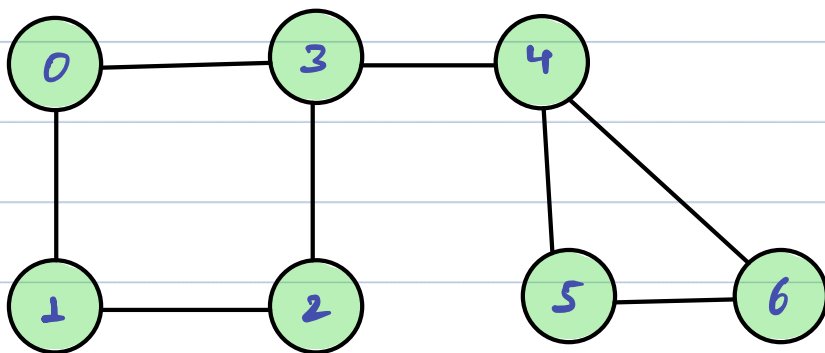
$$\frac{N(N-1)}{2} = \frac{N^2}{2} - \frac{N}{2} = O(N^2)$$

Break till 10:30 pm

→ breadth first search



BFS traversal

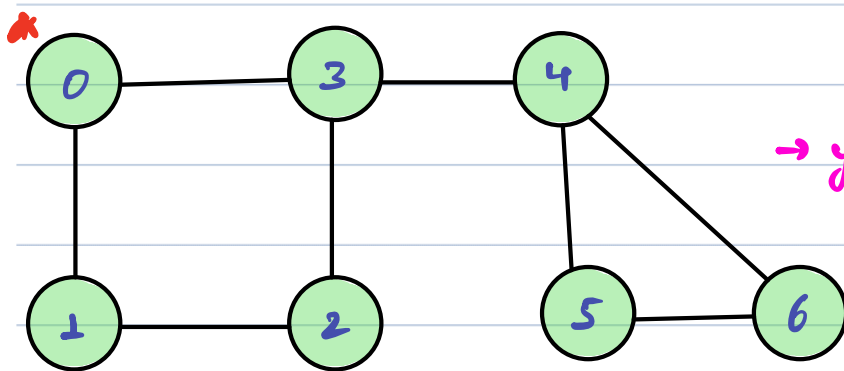
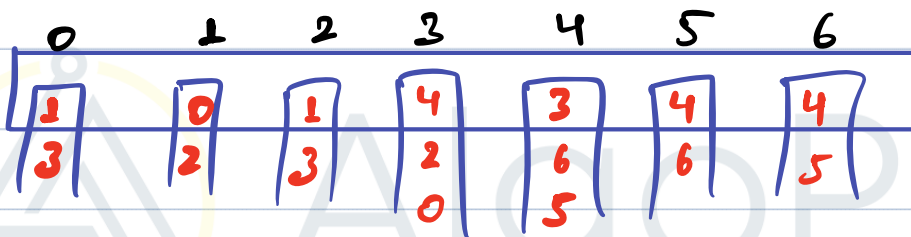


→ no. of nodes
N = 7
→ no. of edges
M = 8

	0	1
0	3	4
1	1	2
2	2	3
3	4	6
4	0	1
5	4	5
6	5	6
	0	3

List < List < Integer > > graph = new ArrayList<>();

graph



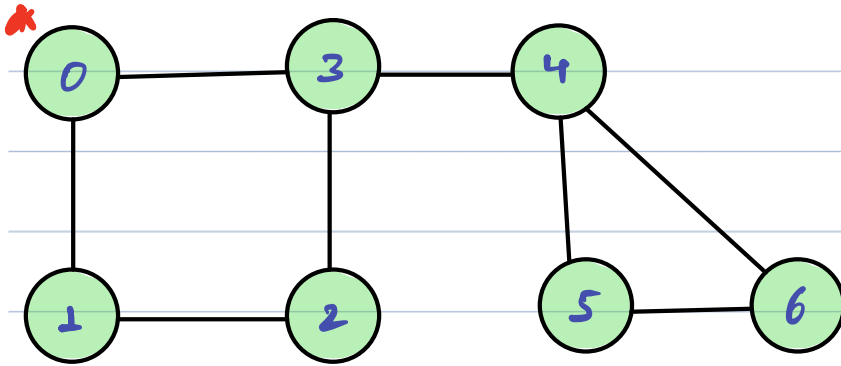
→ you will have to decide the starting point.

(0) (1 3) (4 2) (5 6)



graph

0	1	2	3	4	5	6
1 3	0 2	1 3	4 2 0	3 6 5	4 6	4 5



queue: ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~

vis :

0	1	2	3	4	5	6
T	F	T	F	T	F	T

0 (1 3) (2 4) (6 5)



Pseudocode

no. of nodes
no. of vertices
void BFS (int n , int m , int edges[m][2]) {

List<List<Integer>> graph = construction($n, m, edges$);

Queue<Integer> q;

boolean[] vis = new boolean[n];

q.add(0);

vis[0] = true;

while (q.size() > 0) {

int rem = q.remove();

S.o.p(rem);

// add unvisited nodes

List<Integer> nbs = graph.get(rem);

for (int v: nbs) {

if (vis[v] == false) {

q.add(v);

vis[v] = true;

}

}

}

}

T.C! $\rightarrow$ H.W